AWS Cloud Practitioner

Module -2

- **Compute** – Amazon EC2, Launching EC2 Instances using Putty, Types of Instances, AWS Elastic Beanstalk, Local Zone,
- **Storage** –Amazon EBS, AWS backup – Snapshots, Amazon S3, Storage Class overview, S3 Glacier, and Storege gateway.
- **Serverless** – AWS Lambda and AWS Fargate.

Compute Service – Amazon EC2

Amazon Elastic Compute Cloud (Amazon EC2)

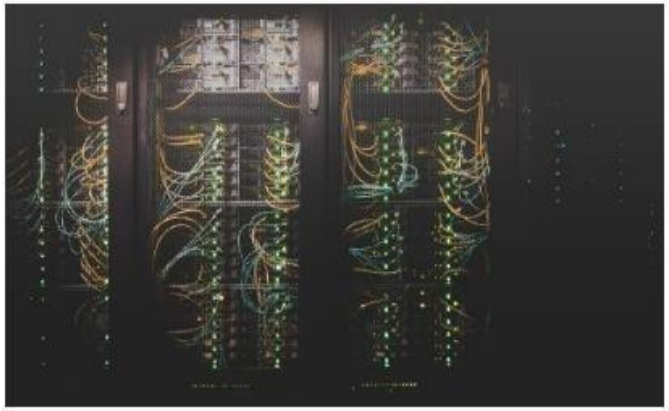
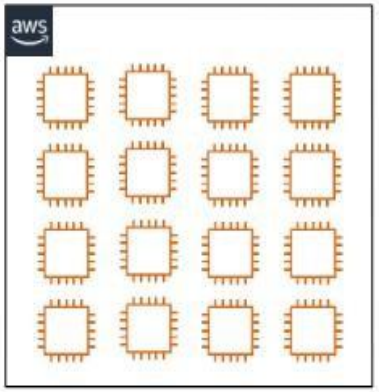


Photo by Taylor Vick on Unsplash

On-premises servers

Example uses of Amazon EC2 instances

- ✓ Application server
- ✓ Web server
- ✓ Database server
- ✓ Game server
- ✓ Mail server
- ✓ Media server
- ✓ Catalog server
- ✓ File server
- ✓ Computing server
- ✓ Proxy server



Amazon EC2 instances



Photo by panumas nikhomkhai from Pexels

Amazon EC2 overview

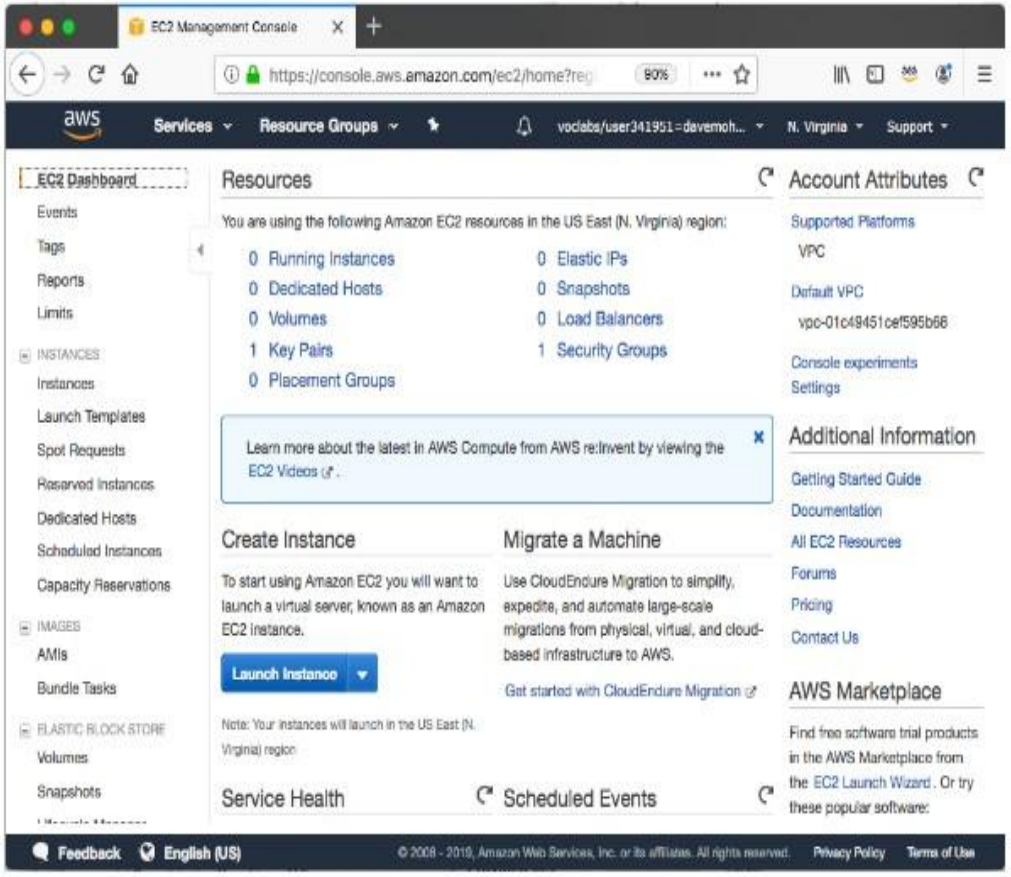


- **Amazon Elastic Compute Cloud (Amazon EC2)**
  - Provides virtual machines—referred to as EC2 instances—in the cloud.
  - Gives you *full control* over the guest operating system (Windows or Linux) on each instance.
- You can launch instances of any size into an Availability Zone anywhere in the world.
  - Launch instances from **Amazon Machine Images (AMIs)**.
  - Launch instances with a few clicks or a line of code, and they are ready in minutes.
- You can control traffic to and from instances.

Launching an Amazon EC2 instance

This section of the module walks through nine key decisions to make when you create an EC2 instance by using the AWS Management Console Launch Instance Wizard.

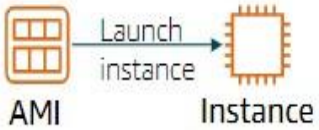
- Along the way, essential Amazon EC2 concepts will be explored.



1. Select an AMI

Choices made using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair



- Amazon Machine Image (AMI)
  - Is a template that is used to create an EC2 instance (which is a **virtual machine, or VM,** that runs in the AWS Cloud)
  - Contains a **Windows** or **Linux** operating system
  - Often also has some **software** pre-installed
- AMI choices:
  - Quick Start – *Linux and Windows AMIs that are provided by AWS*
  - My AMIs – *Any AMIs that you created*
  - AWS Marketplace – *Pre-configured templates from third parties*
  - Community AMIs – *AMIs shared by others; use at your own risk*

2. Select an instance type

Choices made using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Consider your use case
  - How will the EC2 instance you create be used?
- The **instance type** that you choose determines –
  - Memory (RAM)
  - Processing power (CPU)
  - Disk space and disk type (Storage)
  - Network performance



- Instance type categories –
  - General purpose
  - Compute optimized
  - Memory optimized
  - Storage optimized
  - Accelerated computing
- Instance types offer *family*, *generation*, and *size*

EC2 instance type naming and sizes

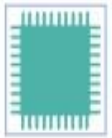
Instance type naming

- Example: **t3.large**
  - T is the family name
  - 3 is the generation number
  - Large is the size

Example instance sizes

Instance Name	vCPU	Memory (GB)	Storage
t3.nano	2	0.5	EBS-Only
t3.micro	2	1	EBS-Only
t3.small	2	2	EBS-Only
t3.medium	2	4	EBS-Only
t3.large	2	8	EBS-Only
t3.xlarge	4	16	EBS-Only
t3.2xlarge	8	32	EBS-Only

Select instance type: Based on use case




	General Purpose	Compute Optimized	Memory Optimized	Accelerated Computing	Storage Optimized
Instance Types	a1, m4, m5, t2, t3	c4, c5	r4, r5, x1, z1	f1, g3, g4, p2, p3	d2, h1, i3
Use Case	Broad	High performance	In-memory databases	Machine learning	Distributed file systems

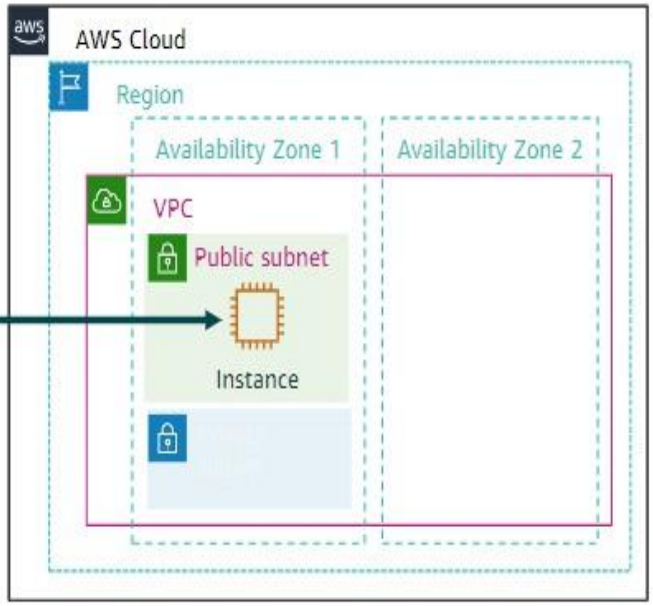
3. Specify network settings

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Where should the instance be deployed?
  - Identify the **VPC** and optionally the **subnet**
- Should a **public IP address** be automatically assigned?
  - To make it internet-accessible

Example: specify to deploy the instance here



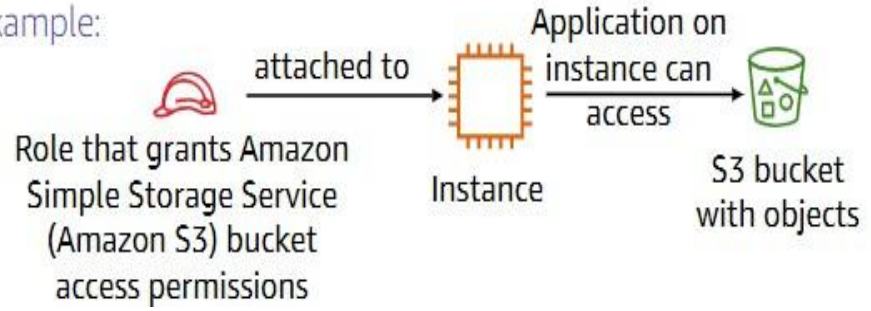
4. Attach IAM role (optional)

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Will software on the EC2 instance need to interact with other AWS services?
  - If yes, attach an appropriate **IAM Role**.
- An AWS Identity and Access Management (IAM) role that is attached to an EC2 instance is kept in an **instance profile**.
- You are *not* restricted to attaching a role only at instance launch.
  - You can also attach a role to an instance that already exists.

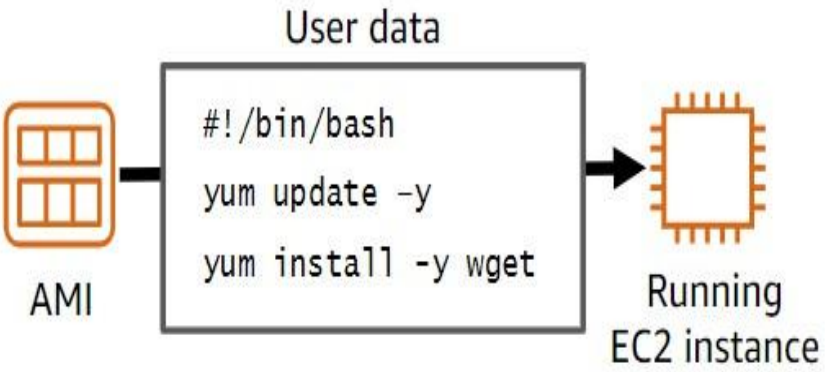
Example:



5. User data script (optional)

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair



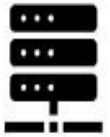
- Optionally specify a user data script at instance launch
- Use **user data** scripts to customize the runtime environment of your instance
  - Script runs the first time the instance starts
- Can be used strategically
  - For example, reduce the number of custom AMIs that you build and maintain

6. Specify storage

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Configure the root volume
  - Where the guest operating system is installed
- Attach additional storage volumes (optional)
  - AMI might already include more than one volume
- For each volume, specify:
  - The **size** of the disk (in GB)
  - The **volume type**
    - Different types of solid state drives (SSDs) and hard disk drives (HDDs) are available
  - If the volume will be deleted when the instance is terminated
  - If **encryption** should be used





Amazon EC2 storage options

- **Amazon Elastic Block Store (Amazon EBS) –**
  - Durable, block-level storage volumes.
  - You can stop the instance and start it again, and the data will still be there.
- **Amazon EC2 Instance Store –**
  - Ephemeral storage is provided on disks that are attached to the host computer where the EC2 instance is running.
  - If the instance stops, data stored here is deleted.
- Other options for storage (not for the root volume) –
  - Mount an **Amazon Elastic File System (Amazon EFS)** file system.
  - Connect to **Amazon Simple Storage Service (Amazon S3)**.

7. Add tags

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- A tag is a label that you can assign to an AWS resource.
  - Consists of a *key* and an optional *value*.
- Tagging is how you can attach **metadata** to an EC2 instance.
- Potential benefits of tagging—Filtering, automation, cost allocation, and access control.

Example:



8. Security group settings

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- A security group is a **set of firewall rules** that control traffic to the instance.
  - It exists *outside* of the instance's guest OS.
- Create **rules** that specify the **source** and which **ports** that network communications can use.
  - Specify the **port** number and the **protocol**, such as Transmission Control Protocol (TCP), User Datagram Protocol (UDP), or Internet Control Message Protocol (ICMP).
  - Specify the **source** (for example, an IP address or another security group) that is allowed to use the rule.

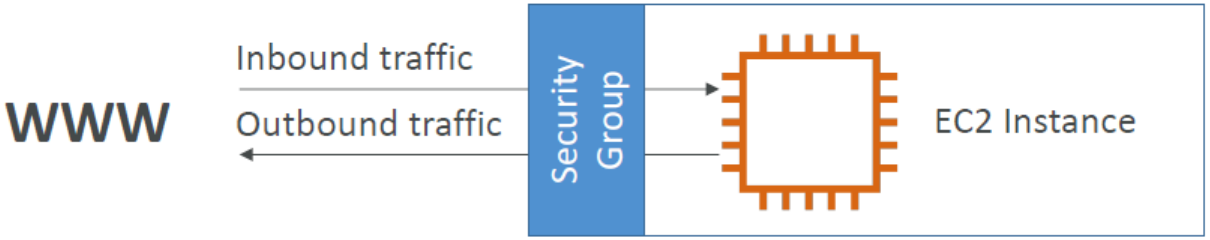
Example rule:



aws

Introduction to Security Groups

- Security Groups are the fundamental of network security in AWS
- They control how traffic is allowed into or out of our EC2 Instances.



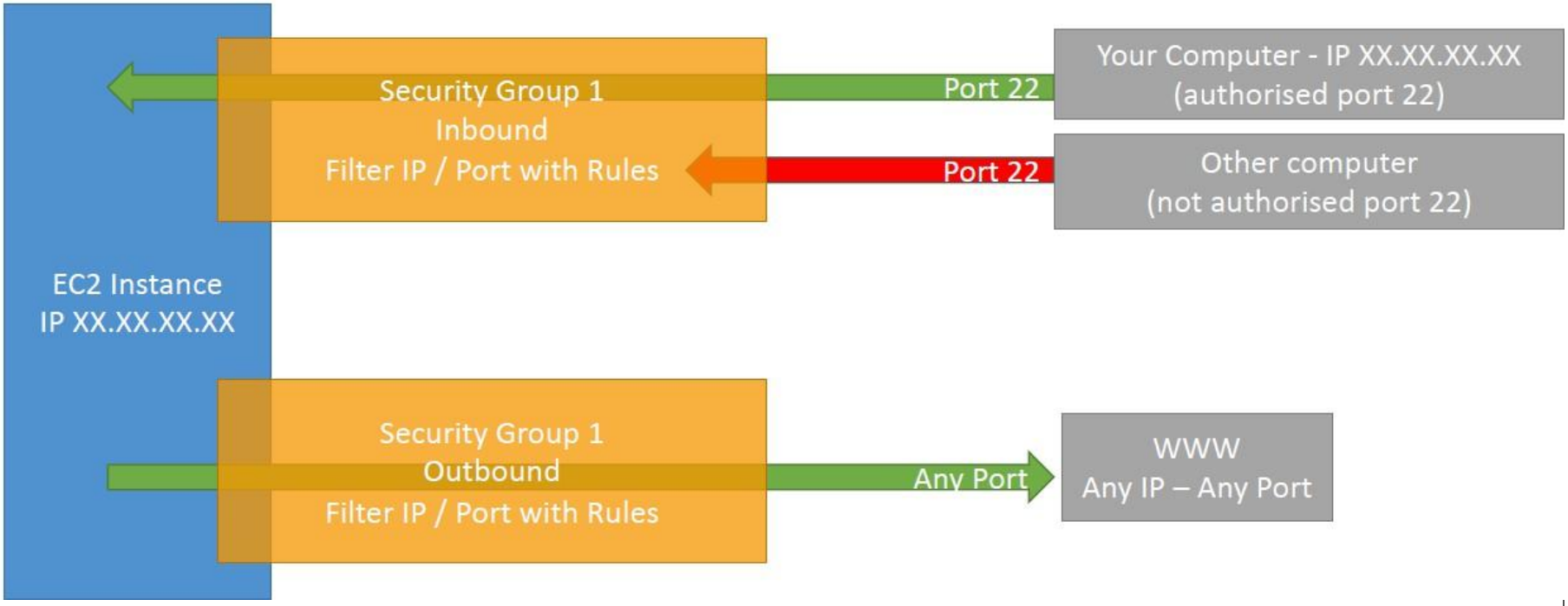
- Security groups only contain allow rules
- Security groups rules can reference by IP or by security group

Security Groups Deeper Dive

- Security groups are acting as a "firewall" on EC2 instances
- They regulate:
  - Access to Ports
  - Authorised IP ranges – IPv4 and IPv6
  - Control of inbound network (from other to the instance)
  - Control of outbound network (from the instance to other)

Type	Protocol	Port Range	Source	Description
HTTP	TCP	80	0.0.0.0/0	test http page
SSH	TCP	22	122.149.196.85/32	
Custom TCP Rule	TCP	4567	0.0.0.0/0	java app

Security Groups Diagram



Classic Ports to know

- 22 = SSH (Secure Shell) - log into a Linux instance
- 21 = FTP (File Transfer Protocol) – upload files into a file share
- 22 = SFTP (Secure File Transfer Protocol) – upload files using SSH
- 80 = HTTP – access unsecured websites
- 443 = HTTPS – access secured websites
- 3389 = RDP (Remote Desktop Protocol) – log into a Windows instance

9. Identify or create the key pair

Choices made by using the Launch Instance Wizard:

1. **AMI**
2. **Instance Type**
3. **Network settings**
4. **IAM role**
5. **User data**
6. **Storage options**
7. **Tags**
8. **Security group**
9. **Key pair**

- At instance launch, you specify an existing key pair *or* create a new key pair.
- A key pair consists of –
  - A **public key** that AWS stores.
  - A **private key** file that you store.
- It enables secure connections to the instance.
- For **Windows AMIs –**
  - Use the private key to obtain the administrator password that you need to log in to your instance.
- For **Linux AMIs –**
  - Use the private key to use SSH to securely connect to your instance.



mykey.pem



Type of Instances

Amazon EC2 (Elastic Compute Cloud) offers a wide variety of instance types tailored to meet different use cases. These instance types are categorized based on their compute, memory, storage, and networking capabilities. Each instance type belongs to a specific **instance family** optimized for particular workloads.

1. General Purpose Instances

- **Balanced compute, memory, and networking resources.**
- Ideal for applications with diverse workloads, such as web servers, app servers, and development environments.
- Examples:
  - **T Series (Burstable Performance)**: t2, t3, t4g
    - Cost-effective for low to moderate workloads.
  - **M Series**: m5, m6g, m7i
    - Balanced performance for a wide range of applications.

Type of Instances

2. Compute-Optimized Instances

- **High-performance processors for compute-intensive tasks.**
- Ideal for high-performance computing (HPC), machine learning inference, batch processing, and gaming.
- Examples:
  - **C Series**: c5, c6g, c7g

3. Memory-Optimized Instances

- **High memory-to-CPU ratio for memory-intensive applications.**
- Ideal for in-memory databases, caching, and real-time big data analytics.
- Examples:
  - **R Series**: r5, r6g, r7g
    - Optimized for memory-intensive applications.
  - **X Series**: x2idn, x2iedn

Type of Instances

4. Accelerated Computing Instances

- **Hardware accelerators like GPUs and FPGAs for specialized tasks.**
- Ideal for machine learning, deep learning, video processing, and scientific simulations.
- Examples:
  - **P Series**: p3, p4 (GPU instances for ML training).
  - **G Series**: g4, g5 (GPU instances for ML inference and graphics rendering).

Type of Instances

5. Storage-Optimized Instances

- **High disk throughput and IOPS for storage-heavy applications.**
- Ideal for data warehousing, distributed file systems, and NoSQL databases.
- Examples:
  - **I Series**: i3, i4i
    - Optimized for high IOPS storage.
  - **D Series**: d2, d3
    - Optimized for dense storage workloads.
  - **H Series**: h1
    - Optimized for high storage throughput.

Type of Instances

6. Networking-Optimized Instances

•Designed for applications requiring high network bandwidth and low latency.

•Examples:

- **High Network Bandwidth Instances**: Instances with enhanced networking capabilities like c5n and m5n.

Elastic Beanstalk



AWS Elastic Beanstalk

- An easy way to get **web applications** up and running
- A **managed service** that automatically handles –
  - Infrastructure provisioning and configuration
  - Deployment
  - Load balancing
  - Automatic scaling
  - Health monitoring
  - Analysis and debugging
  - Logging
- No additional charge for Elastic Beanstalk
  - Pay only for the underlying resources that are used

Key features of Elastic Beanstalk

✅ **Easy Deployment** – Upload your application code, and EB automatically provisions the necessary infrastructure.

✅ **Supports Multiple Languages** – Java, Python, .NET, Node.js, Ruby, PHP, Go, and Docker.

✅ **Integrated Load Balancing & Auto Scaling** – Automatically adjusts resources based on demand.

✅ **Monitoring & Logging** – Built-in integration with **Amazon CloudWatch** and **AWS X-Ray** for performance tracking.

✅ **Customization** – Modify configurations, set environment variables, and extend with EC2 customizations.

How Elastic Beanstalk works?

- When you deploy an application using Elastic Beanstalk, it follows these steps:

1. **Upload Your Code** – Use the AWS Management Console, AWS CLI, or an IDE plugin to upload your application.

2. **Elastic Beanstalk Provisions Infrastructure** – It automatically creates:
    1. EC2 instances
    2. Elastic Load Balancer (ELB)
    3. Auto Scaling Group
    4. Security Groups
    5. Amazon RDS (if configured)

How Elastic Beanstalk works?

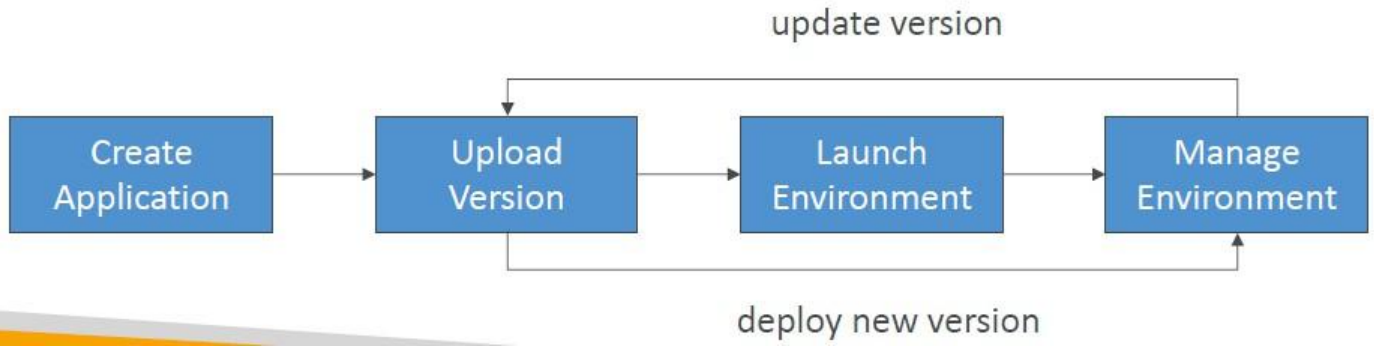
3. Application Deployment – EB configures the environment and deploys your application.

4. Auto-Scaling & Monitoring – The application is monitored, and resources are adjusted based on traffic.

5. Application Updates – You can deploy updates with zero downtime using **Rolling Updates**.

Elastic Beanstalk – Components

- **Application:** collection of Elastic Beanstalk components (environments, versions, configurations, …)
- **Application Version:** an iteration of your application code
- **Environment**
  - Collection of AWS resources running an application version (only one application version at a time)
  - **Tiers:** Web Server Environment Tier & Worker Environment Tier
  - You can create multiple environments (dev, test, prod, …)



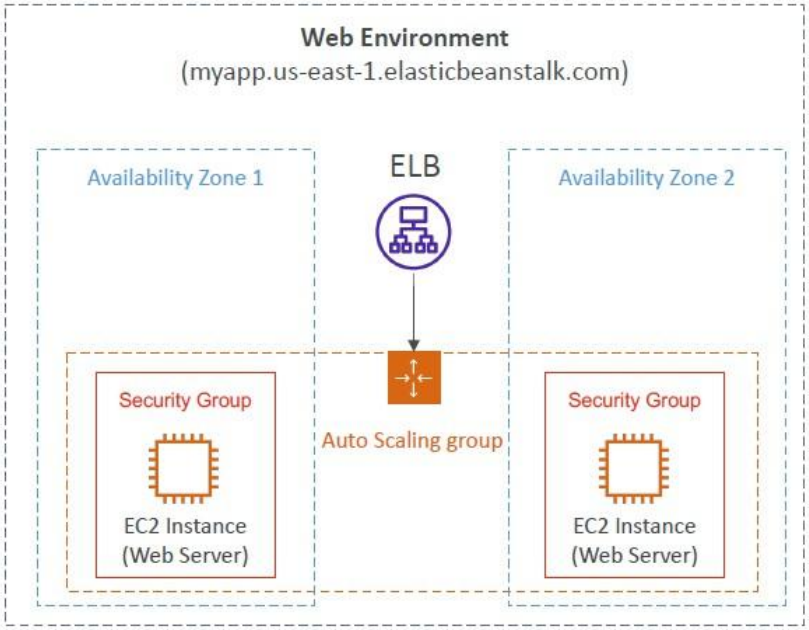
AWS Elastic Beanstalk provides two types of environments for running applications:

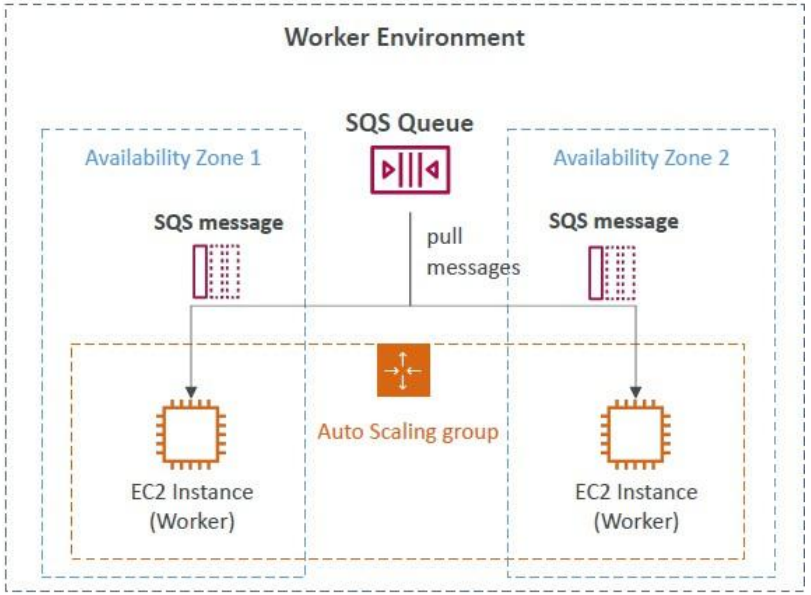
1. **Web Server Environment Tier** – Handles **HTTP(S) requests** for web applications.
2. **Worker Environment Tier** – Processes **background tasks** asynchronously.

Elastic Beanstalk

- Support for many platforms:
  - Go
  - Java SE
  - Java with Tomcat
  - .NET on Windows Server with IIS
  - Node.js
  - PHP
  - Python
  - Ruby
  - Packer Builder
  - Single Container Docker
  - Multi-Container Docker
  - Preconfigured Docker
- If not supported, you can write your custom platform (advanced)

Web Server Tier vs. Worker Tier





- Scale based on the number of SQS messages
- Can push messages to SQS queue from another Web Server Tier

Web Server Tier

- A **Web Server Environment** is designed to run web applications that handle **HTTP(S) requests** from users via an **Elastic Load Balancer (ELB)**.

- **How It Works**

1. Elastic Beanstalk provisions **Amazon EC2 instances**.
2. It configures a **Load Balancer** (ELB) and **Auto Scaling Group**.
3. EC2 instances run a web server (Apache, Nginx, IIS, etc.).
4. The environment automatically manages scaling and monitoring.

- **Use Cases**
- ✅ Hosting **web applications** (Node.js, Django, Spring Boot, Laravel, etc.)
  ✅ Running **REST APIs**
  ✅ Serving **static and dynamic content**

Web Server Tier

- **Architecture**

User Request --> Load Balancer --> EC2 Web Server --> Application Code --> Database (optional)

Worker Tier

- A **Worker Environment** is designed for processing **background jobs** that do not require immediate responses.

- **How It Works**

1. Elastic Beanstalk provisions **EC2 instances** running a worker process.
2. It creates an **Amazon Simple Queue Service (SQS)** queue.
3. The worker tier listens to the SQS queue for tasks.
4. When a task arrives, the worker instance processes it.

- **Use Cases**
- ✅ **Asynchronous task processing** (e.g., sending emails, image processing)
  ✅ **Scheduled jobs** (e.g., database cleanup, report generation)
  ✅ **Long-running operations** (e.g., video encoding, machine learning batch jobs)

Worker Tier

Architecture

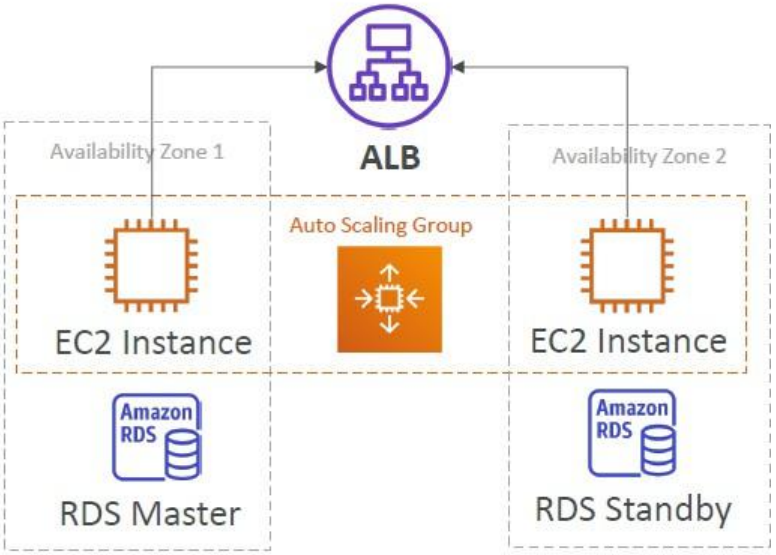
Web Server (User Request) --> SQS Queue --> Worker EC2 Instance --> Task Execution

Elastic Beanstalk Deployment Modes

Single Instance
Great for dev

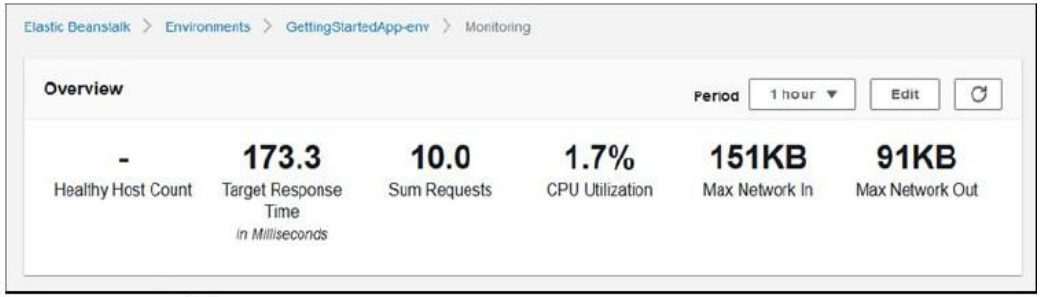


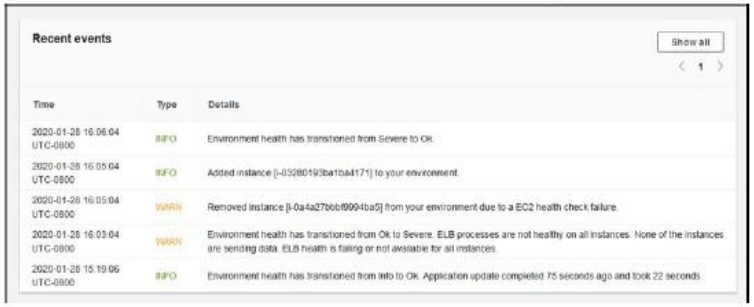
High Availability with Load Balancer
Great for prod

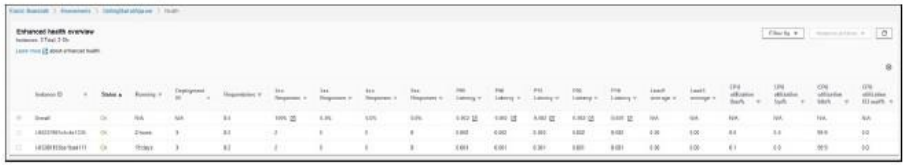


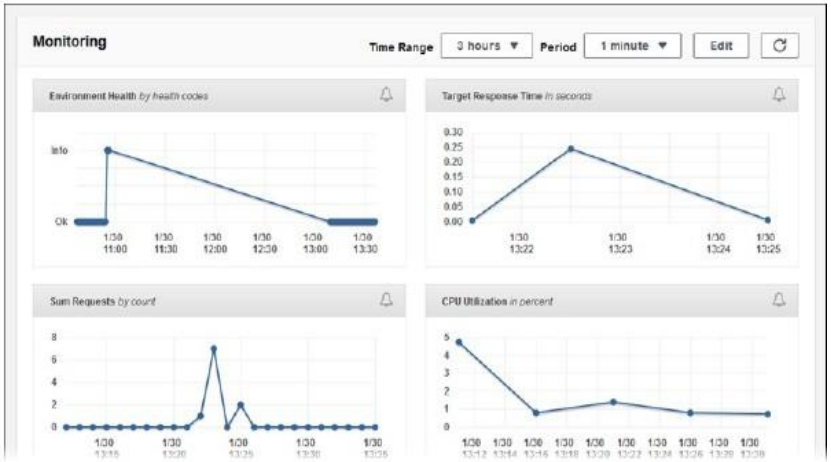
Elastic Beanstalk – Health Monitoring

- Health agent pushes metrics to CloudWatch
- Checks for app health, publishes health events







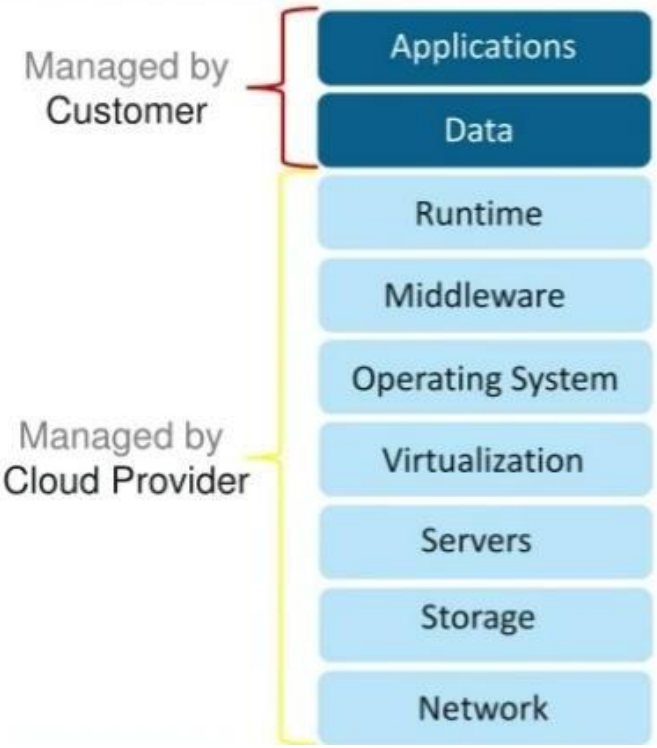


Health Status Levels

Elastic Beanstalk assigns a **color-coded health status** to indicate the state of your environment.

Health Color	Status	Description
🟢 **Green**	OK	Environment is healthy and responding normally.
🟡 **Yellow**	Warning	Some instances are experiencing minor issues.
🔴 **Red**	Severe	One or more instances are failing, affecting application performance.
⚫ **Grey**	No Data	Elastic Beanstalk is unable to retrieve health metrics.

PAAS(Platform-As-A-Service)



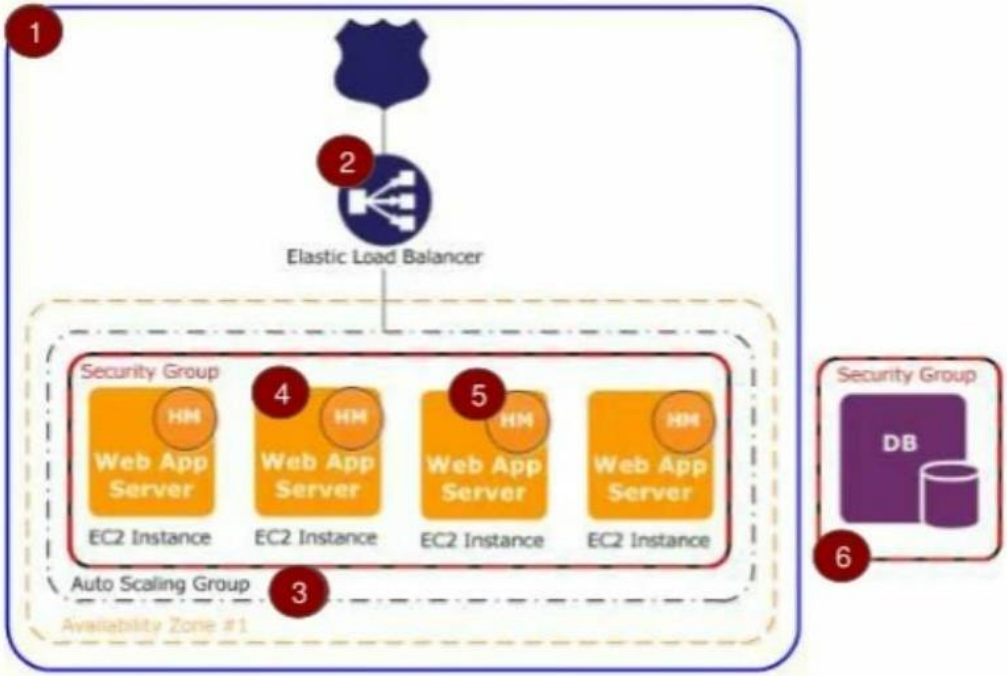


Features of AWS Elastic Beanstalk



Web Server Environment

Web Server Environment Tier handles HTTP requests.



01 Beanstalk Environment

02 Elastic Load Balancer

03 Auto Scaling Group

04 EC2 Instances

05 Host Manager

06 Security Groups

AWS Local Zones

AWS Local Zones are extensions of AWS Regions that bring compute, storage, and other select AWS services closer to major population centers, industry hubs, and end users. They are designed to address use cases requiring low latency (single-digit millisecond) or local data residency.

- **Key Features of AWS Local Zones**

1. **Proximity to End Users**:
    1. Provides access to AWS services closer to users for applications sensitive to latency, like gaming, live video streaming, and real-time simulations.
2. **Reduced Latency**:
    1. By hosting applications closer to users, Local Zones eliminate the latency introduced by connecting to a distant AWS Region.

AWS Local Zones

3. Subset of AWS Services:

•Offers a select set of services like **Amazon EC2**, **Amazon EBS**, **Amazon ECS**, **Amazon EKS**, **AWS Direct Connect**, and **Amazon VPC**.

•The parent AWS Region handles the rest of the services.

4. Seamless Integration:

•Local Zones are extensions of AWS Regions, which means management and deployment workflows remain consistent across the AWS ecosystem.

5. Local Data Residency:

•Helps meet compliance and regulatory requirements for data residency by keeping data within a specific location.

AWS Backup

AWS Backup is a fully managed, centralized service that simplifies and automates the process of backing up data across AWS services and on-premises environments.

It provides a unified approach to data protection, enabling you to define backup policies, monitor backup activity, and ensure data compliance from a single platform.

- **Key Features of AWS Backup**

1. **Centralized Backup Management**
    1. Manage backups for various AWS services from a single console or API.
    2. Supports data stored in services like Amazon EBS, Amazon RDS, Amazon DynamoDB, Amazon EFS, AWS Storage Gateway, and more.

AWS Backup

2. Policy-Based Automation

•Define backup schedules, retention periods, and lifecycle policies using backup plans.

•Automatically apply policies to resources based on tags.

3. Secure and Compliant Backups

•Supports encryption of data at rest and in transit.

•Helps meet regulatory and compliance requirements like GDPR, HIPAA, and PCI DSS.

4. Cross-Region and Cross-Account Backups

•Replicate backups across AWS regions and accounts for disaster recovery and improved resilience.

AWS Backup

5. Backup Monitoring and Alerts

•Use Amazon CloudWatch and AWS Backup Audit Manager to monitor backup activity.

•Receive alerts for backup status and compliance violations.

6. Lifecycle Management

•Automatically transition backups to cold storage after a specified period, reducing storage costs.

•Retain backups for as long as required based on regulatory or business needs.

7. Point-in-Time Recovery

•Recover data to a specific point in time (supported for certain services like RDS and DynamoDB).

Supported AWS Services

- **Amazon EBS**: Backups for block storage volumes.
- **Amazon RDS**: Automated backups for relational databases.
- **Amazon DynamoDB**: Point-in-time recovery for NoSQL databases.
- **Amazon EFS**: Backups for file systems.
- **AWS Storage Gateway**: Protects data stored on-premises and in AWS.
- **Amazon S3** (coming soon): Manage S3 backups using AWS Backup.

Storage-Amazon EBS

AWS Storage Services: AWS offers a wide range of storage services that can be provisioned depending on your project requirements and use case.

AWS storage services have different provisions for highly confidential data, frequently accessed data, and the not so frequently accessed data.

You can choose from various storage types namely, object storage, file storage, block storage services, backups,, and data migration options. All of which fall under the AWS Storage Services list.

Storage-Amazon EBS

- **Elastic Block Storage (EBS):** From the aforementioned list, EBS is a block type durable and persistent storage that can be attached to EC2 instances for additional storage. Unlike EC2 instance storage volumes which are suitable for holding temporary data EBS volumes are highly suitable for essential and long-term data. EBS volumes are specific to availability zones and can only be attached to instances within the same availability zone.
- EBS can be created from the EC2 dashboard in the console as well as in Step 4 of the EC2 launch. Just note that when creating EBS with EC2, the EBS volumes are created in the same availability zone as EC2, however when provisioned independently users can choose the AZ in which EBS is required.

Storage-Amazon EBS

- **Features of EBS:**

- **Scalability:** EBS volume sizes and features can be scaled as per the needs of the system. This can be done in two ways:

  - Take a snapshot of the volume and create a new volume using the Snapshot with new updated features.
  - Updating the existing EBS volume from the console.

Storage-Amazon EBS

- **Features of EBS (contd..**

- **Backup:** Users can create **snapshots** of EBS volumes that act as backups.

  - Snapshot can be created manually at any point in time or can be scheduled.
  - Snapshots are stored on AWS S3 and are charged according to the S3 storage charges.
  - Snapshots are incremental in nature.
  - New volumes across regions can be created from snapshots.

Storage-Amazon EBS

- **Features of EBS (contd..**

- **Encryption:** Encryption can be a basic requirement when it comes to storage. This can be due to the government of regulatory compliance. EBS offers an **AWS managed encryption feature.**

  - Users can enable encryption when creating EBS volumes by clicking on a checkbox.
  - Encryption Keys are managed by the Key Management Service (KMS) provided by AWS.
  - Encrypted volumes can only be attached to selected instance types.
  - Encryption uses the AES-256 algorithm.
  - Snapshots from encrypted volumes are encrypted and similarly, volumes created from snapshots are encrypted.

Storage-Amazon EBS

- **Features of EBS (contd..**

- **Charges:** Unlike AWS S3, where you are charged for the storage you consume, AWS charges users for the storage you hold. For example if you use 1 GB storage in a 5 GB volume, you'd still be charged for a 5 GB EBS volume.
  - EBS charges vary from region to region.
- EBS Volumes are independent of the EC2 instance they are attached to. The data in an EBS volume will remain unchanged even if the instance is rebooted or terminated.
- Single EBS volume can only be attached to one EC2 instance at a time. However, one EC2 can have more than one EBS volumes attached to it.

Types of EBS

1. General Purpose SSD (gp3, gp2):

•Cost-effective and suitable for a variety of workloads, including boot volumes and low-latency applications.

2. Provisioned IOPS SSD (io2, io1):

•Designed for latency-sensitive transactional workloads, such as databases.

3. Throughput Optimized HDD (st1):

•Ideal for sequential data access workloads like big data, data warehouses, and log processing.

4. Cold HDD (sc1):

•Lowest cost; suitable for infrequently accessed data.

5. EBS Magnetic (Standard):

•Legacy option for workloads that do not require high performance.

EBS – Use cases

- Hosting relational and non-relational databases.
- Storing logs and data files for big data analytics.
- Running applications that require high availability and reliability.
- Supporting disaster recovery and business continuity through snapshots and cross-region replication.

AWS Snapshots

•In AWS, **snapshots** are point-in-time backups of data, typically associated with storage services like Amazon Elastic Block Store (EBS), Amazon RDS, and Amazon FSx. Snapshots are widely used for data protection, disaster recovery, and migration purposes.

- **Key Features of Snapshots**

1. **Incremental Backups**
    1. After the first full snapshot, subsequent snapshots are incremental, meaning only changes since the last snapshot are stored. This reduces storage costs and speeds up the backup process.
2. **Cross-Region and Cross-Account Copying**
    1. Snapshots can be copied to different AWS regions for disaster recovery or closer proximity to end-users.
    2. They can also be shared with other AWS accounts securely.

AWS Snapshots

3.Encryption

•Snapshots of encrypted volumes or databases are also encrypted.

•Data is protected using AWS Key Management Service (KMS).

4.Lifecycle Management

•Automate snapshot creation, retention, and deletion using AWS Backup or Data Lifecycle Manager (DLM).

5.Fast Recovery

•Snapshots can be used to quickly restore volumes or databases to a specific point in time.

Types of AWS Snapshots

1. EBS Snapshots

- Create backups of Amazon EBS volumes.
- Can be restored to create a new volume or attached to an instance.
- Support fast snapshot restore (FSR) for selected volumes, reducing recovery times.

2. RDS Snapshots

- Back up Amazon RDS databases.
- Two types:
  - **Automated Snapshots**: Automatically created based on retention settings.
  - **Manual Snapshots**: User-initiated and retained until explicitly deleted.
- Used for point-in-time recovery.

Types of AWS Snapshots

3. DynamoDB Backups

- On-demand snapshots or continuous backups for point-in-time recovery.

4. Amazon FSx Snapshots

- Backup file systems like FSx for Windows, FSx for Lustre, or FSx for OpenZFS.
- Ensures consistent recovery for shared file storage.

5. Amazon Redshift Snapshots

- Backup data in Amazon Redshift clusters.
- Can be automated or manual.

AWS Snapshots

Use Cases

1. **Disaster Recovery**
    1. Maintain backups in different regions to ensure availability during outages.
2. **Migration**
    1. Use snapshots to migrate volumes or databases across accounts or regions.
3. **Development and Testing**
    1. Create snapshots of production data for non-production environments.
4. **Data Retention and Archiving**
    1. Preserve snapshots for compliance and long-term retention.

AWS Simple Storage Service (S3)

Amazon S3 is a Simple Storage Service in AWS that stores files of different types like Photos, Audio, and Videos as Objects providing more scalability and security to.

It allows the users to store and retrieve any amount of data at any point in time from anywhere on the web.

It facilitates features such as extremely high availability, security, and simple connection to other AWS Services.

S3 used for?

- Amazon S3 is used for various purposes in the Cloud because of its robust features with scaling and Securing of data. It helps people with all kinds of use cases from fields such as Mobile/Web applications, Big data, Machine Learning and many more. The following are a few Wide Usage of Amazon S3 service.
- **Data Storage:** Amazon s3 acts as the best option for scaling both small and large storage applications. It helps in storing and retrieving the data-intensitive applications as per needs in ideal time.
- **Backup and Recovery:** Many Organizations are using Amazon S3 to backup their critical data and maintain the data durability and availability for recovery needs.

S3 used for?

- **Hosting Static Websites:** Amazon S3 facilitates in storing HTML, CSS and other web content from Users/developers allowing them for hosting Static Websites benefiting with low-latency access and cost-effectiveness. To know more detailing refer this Article – How to host static websites using Amazon S3

- **Data Archiving:** Amazon S3 Glacier service integration helps as a cost-effective solution for long-term data storing which are less frequently accessed applications.

- **Big Data Analytics:** Amazon S3 is often considered as data lake because of its capacity to store large amounts of both structured and unstructured data offering seamless integration with other AWS Analytics and AWS Machine Learning Services.

S3 used for?

- **Amazon S3 Bucket:** Data, in S3, is stored in containers called *buckets*. Each bucket will have its own set of policies and configurations.
- This enables users to have more control over their data. Bucket Names must be unique.
- There is a limit of 100 buckets per AWS account. But it can be increased if requested by AWS support.

S3 used for?

- **Amazon S3 Objects:** Fundamental entity type stored in AWS S3.You can store as many objects as you want to store. The maximum size of an AWS S3 bucket is 5TB. It consists of the following:

- Key
- Version ID
- Value
- Metadata
- Subresources
- Access control information
- Tags

Amazon S3 Versioning and Access Control

- **S3 Versioning:** Versioning means always keeping a record of previously uploaded files in S3. Points to Versioning are not enabled by default. Once enabled, it is enabled for all objects in a bucket. Versioning keeps all the copies of your file, so, it adds cost for storing multiple copies of your data. For example, 10 copies of a file of size 1GB will have you charged for using 10GBs for S3 space. Versioning is helpful to prevent unintended overwrites and deletions. Objects with the same key can be stored in a bucket if versioning is enabled (since they have a unique version ID).
- **Access control lists (ACLs)**: A document for verifying access to S3 buckets from outside your AWS account. An ACL is specific to each bucket. You can utilize S3 Object Ownership, an Amazon S3 bucket-level feature, to manage who owns the objects you upload to your bucket and to enable or disable ACLs.

Amazon S3 Storage Classes

- Amazon S3 Standard
- Amazon S3 Intelligent-Tiering
- Amazon S3 Standard-Infrequent Access
- Amazon S3 One Zone-Infrequent Access
- Amazon S3 Glacier Instant Retrieval
- Amazon S3 Glacier Flexible Retrieval
- Amazon S3 Glacier Deep Archive

Amazon S3 Storage Classes

- **Amazon S3 Standard**

- It is used for general purposes and offers high durability, availability, and performance object storage for frequently accessed data. S3 Standard is appropriate for a wide variety of use cases, including cloud applications, dynamic websites, content distribution, mobile and gaming applications, and big data analytics.

- Mainly it is used for general purposes in order to maintain durability, availability, and performance to a higher extent. Its applications are cloud applications, dynamic websites, content distribution, mobile & gaming apps as well as big data analysis or data mining.

- **Characteristics of S3 Standard**
- Availability criteria are quite good like 99.9%.
- Improves the recovery of an object file.

Amazon S3 Storage Classes

- **Amazon S3 Intelligent-Tiering**

- The first cloud storage automatically decreases the user's storage cost. It provides very cost-effective access based on frequency, without affecting other performances. It also manages tough operations. Amazon S3 Intelligent – Tiering reduces the cost of granular objects automatically. No retrieval charges are there in Amazon S3 Intelligent – Tiering.

- **Characteristics of S3 Intelligent-Tiering**

- Required less monitoring and automatically tier charge.
- No minimum storage duration and no recovery charges are required to access the service.
- Availability criteria are quite good like 99.9%.
- Durability of S3 Intelligent- Tiering is 99.999999999%.

Amazon S3 Storage Classes

- **S3 Standard-Infrequent Access: Cost-Effective Storage for Less Frequently Used Data**
- To access the less frequently used data, users use S3 Standard-IA. It requires rapid access when needed. We can achieve high strength, high output, and low bandwidth by using S3 Standard-IA. It is best in storing the backup, and recovery of data for a long time. It act as a data store for disaster recovery files.

- **Characteristics of S3 Standard-Infrequent Access**

- High performance and same action rate.
- Very Durable in all AZs.
- Availability is 99.9% in S3 Standard-IA.
- Durability is of 99.999999999%.

Amazon S3 Storage Classes

- **S3 Glacier Instant Retrieval: High-Performance Archiving with Rapid Retrieval**
- It is an archive storage class that delivers the lowest-cost storage for data archiving and is organized to provide you with the highest performance and with more flexibility. S3 Glacier Instant Retrieval delivers the fastest access to archive storage. Same as in S3 standard, Data retrieval in milliseconds .
- **Characteristics of S3 Glacier Instant Retrieval**
- It just takes milliseconds to recover the data.
- The minimum object size should be 128KB.
- Availability is 99.9% in S3 glacier Instant Retrieval.
- Durability is of 99.999999999%.

Amazon S3 Storage Classes

- **S3 One Zone-Infrequent Access: Cost-Optimized Storage for Single Availability Zone**
- Different from other S3 Storage Classes which store data in a minimum of three Availability Zones, S3 One Zone-IA stores data in a single Availability Zone and costs 20% less than S3 Standard-IA. It's a very good choice for storing secondary backup copies of on-premises data or easily re-creatable data. S3 One Zone-IA provides you the same high durability, high throughput, and low latency as in S3 Standard.

- **Characteristics of S3 One Zone-Infrequent Access**

- Supports SSL(Secure Sockets Layer) for data in transferring and encryption of data.
- Availability Zone destruction can damage the data.
- Availability is 99.5% in S3 one Zone- Infrequent Access.
- Durability is of 99.999999999%.

Amazon S3 Storage Classes

- **S3 Glacier Flexible Retrieval: Balancing Cost and Retrieval Flexibility for Archiving**
- It provides low-cost storage compared to S3 Glacier Instant Retrieval. It is a suitable solution for backing up the data so that it can be recovered easily a few times in a year. It just takes minutes to access the data.

- **Characteristics of S3 Glacier Flexible Retrieval**

- Free recoveries in high quantity.
- AZs destruction can lead to difficulty in accessing data.
- when you have to retrieve large data sets , then S3 glacier flexible retrieval is best for backup and disaster recovery use cases.
- Availability is 99.99% in S3 glacier flexible retrieval.
- Durability is of 99.999999999%.

Amazon S3 Storage Classes

- **Amazon S3 Glacier Deep Archive**
- The Glacier Deep Archive storage class is designed to provide long-lasting and secure long-term storage for large amounts of data at a price that is competitive with off-premises tape archival services that is very cheap. You no longer need to deal with expensive services. Accessibility is very much efficient, that it can restore data within 12 hours.

- **Characteristics of S3 Glacier Deep Archive**

- More secured storage.
- Recovery time is less requires less time.
- Availability is 99.99% in S3 glacier deep archive.
- Durability is of 99.999999999%.

AWS Lambda Overview

- **AWS Lambda** is a serverless computing service provided by Amazon Web Services (AWS). It allows you to run code without provisioning or managing servers. Lambda automatically scales and executes your code in response to events, and you only pay for the compute time used.
- **Key Features**

1. **Serverless Architecture**:
    1. No need to manage infrastructure; AWS handles the backend provisioning.
    2. Focus on writing code rather than server setup or maintenance.
2. **Event-Driven Execution**:
    1. Triggers include AWS services (e.g., S3, DynamoDB, API Gateway), HTTP requests, or custom events.
    2. Example: Run code when a file is uploaded to an S3 bucket.

AWS Lambda Overview

3. Automatic Scaling:

1. Automatically handles the number of instances based on the volume of incoming requests.
2. Each function invocation is independent, ensuring high concurrency.

4. Support for Multiple Languages:

1. Native support for Python, Java, Node.js, Ruby, Go, .NET Core, and custom runtimes (via Amazon Linux).
2. Easily extendable using container images.

5. Pay-As-You-Go Pricing:

1. Billed based on the number of requests and execution duration (rounded to the nearest millisecond).
2. Free tier includes 1 million requests and 400,000 GB-seconds per month.

AWS Lambda Overview

6. Integration with AWS Ecosystem:

1. Seamless integration with services like S3, DynamoDB, SNS, SQS, CloudWatch, and more.
2. Use as part of workflows in Step Functions or as a backend for APIs using API Gateway.

7. Security:

1. Managed through AWS IAM for fine-grained permissions.
2. Supports VPC access for private networking.

8. High Availability:

1. Runs in multiple Availability Zones for redundancy and fault tolerance.

AWS Lambda: Advantage

- The following are the advantages of AWS Lambda function

1. **Zero Server Management:** Since AWS Lambda automatically runs the user's code, there's no need for the user to manage the server. Simply write the code and upload it to Lambda.

2. **Scalability:** AWS Lambda runs code in response to each trigger, so the user's application is automatically scaled. The code also runs in parallel processes, each triggered individually, so scaling is done precisely with the size of the workload.

3. **Event-Driven Architecture:** AWS Lambda function can be triggered based on the events happing in the aws other service like when the file or video is added to the S3 bucket you can trigger the AWS Lambda function.

AWS Lambda: Advantage

4. **Automatic High Availability:** When there is high demand or high incoming traffic aws lambda function will automatically scale the servers.

5. **Affordable:** With AWS Lambda, one doesn't pay anything when the code isn't running. The user has to only be charged for every 100ms of code execution and the number of times his code is actually triggered.

AWS Fargate

AWS Fargate is a serverless compute engine for containers offered by Amazon Web Services (AWS). It allows you to run containers without the need to provision, manage, or scale the underlying virtual machines or clusters of EC2 instances. Fargate is tightly integrated with **Amazon Elastic Container Service (ECS)** and **Amazon Elastic Kubernetes Service (EKS)**.

- **Key Features**

1. **Serverless Container Management**:
    1. No need to manage servers, clusters, or EC2 instances.
    2. Focus on defining and deploying your containers.
2. **Seamless Scaling**:
    1. Automatically scales resources to match your container workloads.
    2. Eliminates the need for manual scaling or overprovisioning.

AWS Fargate

3. **Resource Isolation**:
    1. Provides each container with its own compute resources.
    2. Enhances security and performance by avoiding resource contention.
4. **Flexible Resource Configurations**:
    1. Specify CPU and memory independently for each task or pod.
    2. Pay only for the resources your containers use.
5. **Integration with ECS and EKS**:
    1. Supports orchestration through both ECS and Kubernetes (via EKS).
    2. Compatible with existing ECS task definitions or Kubernetes manifests.

AWS Fargate

6. Pay-as-You-Go Pricing:

1. Billed based on the vCPU and memory resources consumed by your containers.
2. No upfront costs or long-term commitments.

7. Networking and Security:

1. Supports **VPC networking**, allowing fine-grained control over container communication.
2. Integrated with AWS Identity and Access Management (IAM) for secure access.
3. Each task or pod gets its own Elastic Network Interface (ENI).

8. High Availability:

1. Runs workloads across multiple Availability Zones for redundancy.

AWS Fargate: Benefits

1.No Infrastructure Management:

•Removes the operational overhead of managing EC2 instances or clusters.

2.Enhanced Security:

•Containers are isolated at the infrastructure level.

•Tasks/pods run in separate ENIs, ensuring network isolation.

3.Cost Efficiency:

•Pay only for the compute and memory resources consumed.

•Reduces costs by eliminating the need to overprovision resources.

AWS Fargate: Benefits

4.Improved Developer Productivity:

•Developers focus on building and deploying applications, not managing servers.

5.Deep Integration with AWS Ecosystem:

•Works seamlessly with AWS services like IAM, CloudWatch, ECR, and more.

Comparison: AWS Fargate vs. Other Options

Feature	Fargate	EC2 (ECS/EKS)	Lambda
Infrastructure Management	Fully managed (serverless)	Requires managing EC2 instances	Fully managed (serverless)
Scaling	Automatic, per task/pod	Manual or auto-scaling groups	Automatic, per function
Use Cases	Containers	Containers	Functions
Billing	Per second (vCPU/memory)	Instance-based (hourly)	Per request and duration
Complexity	Low	Medium	Low